

CS 12 Cheatsheet

Diogn Lei R. Mortera

202508438 · Seat [____] · TCD/FRU2

Asynchronous Programming

blocking operations done sequentially

thread of execution sequence of instructions executed one after another, 1 program - 1 thread (default), web browsers usually have 1 thread

optimization multiple requirements at the same time, related to...

nonblocking operations lets other tasks run even if its not finished

coroutines capable of un/pausing own execution

concurrency the interweaving of coroutines

event loop inf. loop managing concurrent execution of coroutines

async functions returns `Awaitable[...]`, `...` that must be awaited to be executed

serialization complex data to string format (e.g. dict -> JSON), `json.dumps()`

deserialization string format to complex data (e.g. JSON -> dict), `json.loads()`

application programming interface some form of protocol managing communication between sender and receiver

```
import httpx
async def #makes a function a coroutine
await #makes sure a coroutine finishes at that line before proceeding to the next parts of that stack frame
import asyncio
.run(f()) | .sleep(n) |
.gather(f(),g()) # runs all at same time, waits for all to finish
.create_task(f()) # starts running task f() in the background

async with httpx.AsyncClient() as client:
    res = await client.get(f'{API_ENDPOINT}/{id&etc.}', params=params)

JSON null -> Python None
# if not valid JSON,
json.decoder.JSONDecodeError
# if value cannot be serialized to JSON,
TypeError
```

Elm

Basics

Evan Czaplicki, 2012, functional programming for front-end web apps, purely immutable, statically typed, safe type system, no runtime errors

miscellaneous no loops, use recursion, exploit `(List|String).map`

variables must start lowercase, and not indented, no reassignment

statement is not a valid function return value

expression evaluates to value

shadowing reuse of a variable in some declaration, will just give you a warning *but the code still runs so be careful!*

type inferring works, but prefer to type-hint stuff for easier debugging

immutability of containers since variables / values / containers cannot be mutated, nothing is added or removed; thats an entirely new value

Fold

fold abstraction of for-each aggregation pattern (also known as reduce, accumulate, aggregate)

```
sum = List.foldl (+) 0 [1,2,3]
reverse = List.foldl (::) [] [1,2,3,4,5,6]
```

Algebraic Data Types

type alias gives new name to existing type

composite type type composed of other types (tuple, list, array, dict)

product type (and) and **sum type** (or) and **algebraic data type** (composition of both)

tagged unions union type with tag, identifying which type is in use

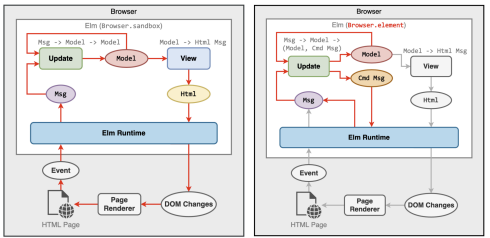
tag data used to identify which variant a tagged union value is

extensible records typing for “record with at least some fields”

```
-- creation of new type
type Color = R | O | Y | G | B | I | V
-- type alias
type alias Name = String
type alias Point2D = { x:Float, y:Float }
-- composite
Bullet = (Point2D, Color)
Magazine = List Bullet
-- product type
type Gun = { mags : Magazine, name : Name }
-- tagged union
type Genre =
    OPM String | KPop String Int | EDM String Float
type Maybe a = Just a | Nothing
type Result e a = Ok a | Err e
-- extensible records: `greet` takes in any record with those attributes
greet : {r | name : String} -> String
```

MVU

model-view-update



C

variables cannot be used when undeclared | cannot be redefined *in the same scope* | contain garbage when not assigned | `snake_case`

initialization initialization = declaration + assignment | `const` variables must be initialized

top level source code outside any block

global variable declared in top level

local variable declared inside some block | usage valid if declaration is in enclosing block

preprocessor directives construct that specifies how a compiler should process its input (`define`, `ifdef`, `ifndef`)

staying within data type limits algebraic manipulation | typecast to wider type

Signed Type	Specifier	Unsigned Type	Specifier
char	%hhc	unsigned char	%hhu
short	%hd	unsigned short	%hu
int	%d	unsigned int	%u
long	%ld	unsigned long	%lu
long long	%lld	unsigned long long	%llu
ssize_t	%zd	size_t	%zu
int8_t	%" PRIi8	uint8_t	%" PRIu8
int16_t	%" PRIi16	uint16_t	%" PRIu16

Signed Type	Specifier	Unsigned Type	Specifier
int32_t	%" PRId32	uint32_t	%" PRIu32
int64_t	%" PRId64	uint64_t	%" PRIu64

change `d` or `u` above to `x` for lowercase hex, and `X` for uppercase hex

sequence points (`;`, `,` && `?:` `f`) if variable modified twice within a sequence, then it is undefined behavior

Memory

memory array of bytes | usually 2^{64} | all data of a running program | “address-value dict”

Heisenbug behavior changes when “observed” (or when something unrelated is added)

null pointer special address value (usually `0`) representing absence of value being pointed to | `NULL` | `#include <stdlib.h>`

segmentation fault program attempts to access memory address it has no permission to

overflow the desired number gets “modulo-ed” under the limits of current datatype

Pointers

Behavior

- when a function call is done, allocated memory is *deallocated*

Rules

- never return pointers to local variables
- `free()` unused memory
- watch out for array out-of-bounds errors
- when creating arrays inside blocks to return, use heap memory (`malloc()`) instead of stack-based
- you cannot dereference a null pointer

Dynamic Memory Allocation

stack memory region for call stack and local vars | faster to access | allocations active until end of function; deallocated automatically

heap memory for dynamic allocation | slower to access | “unlimited” | allocate with `malloc(n*size)` or `calloc(n, size)` | manual deallocation via `free(<ptr>)`

Bugs

- Use-after-free (UAF)
- Double free (cause of security bugs) (CVE-2025-3066, CVE-2023-42950)
- Memory leak (heap memory that can’t be deallocated due to losing all references to it)

Types of Allocations

- Single continuous allocations - exploiting math to index
- Separate contiguous allocations - exploiting nested pointers

Multithreaded Programming

concurrency multiple tasks in given span of time with possible interleaving vs. **parallelism** multiple tasks at same time | multicore processor

thread current instruction + call stack + environment | single core

shared-state concurrency communicate via shared memory

message-passing concurrency communicate via messages (out of scope)

race condition behavior depends on nondeterministic order of thread execution

critical section region with possible race condition

mutual exclusion at most one thread executing critical section at any time

lock/mutex enforces mutual exclusion | lock only acquired if free/released | ≤ 1 thread per lock | can introduce additional overhead, try to minimize

minimize un/locking operations summarizing each thread before updating global variable

